

Second Individual Assessment

 Published

 Assign To

 Edit



COMS W4156 Advanced Software Engineering (ASE)

Second Individual Assessment Fall 2024

Academic Honesty policy: This is an individual assignment. You are not permitted to discuss or work together on the assessment with other students in the class or with any other humans or AI. If you have questions, you may ask the teaching staff, but you may not ask your teammates, other people, search engines, internet forums or AI. Apparent plagiarism, collusion or use of AI (except as explicitly requested in an assessment question) will result in zero credit on the assessment and referral to the relevant disciplinary committee.

Submission policy: Submit your answers all in a single pdf document on courseworks. You can submit as many times as you want until the deadline. Submissions via email, edstem, paper, or any other means besides courseworks will not be graded and will receive zero credit.

Late policy: Submissions posted one day late (which includes everything from 1 minute late to 23 hours 59 minutes late) will receive maximum 75% credit; two days late maximum 50% credit; three days late maximum 25% credit; four days late 0 credit. Note courseworks treats exactly on time as "0 minutes late", but in this case the human graders will recognize that you posted your submission on time.

Maximum total 100 points. There is no credit for part 0, but 0A is **required** in the sense that we will not grade part 1 (the graded questions) if part 0A is missing or incomplete, and you will receive zero credit overall for the assessment. Part 0B is optional and will not affect your grade.

Ungraded questions: part 0A is required and part 0B is optional.

part 0A (required): Allocate 100 total points among the members of your team, *including yourself*. Assign some number of points (possibly 0) to each member of your team based on what you believe was that student's relative participation in the overall team project, and explain your scores. Make sure the total is exactly 100. For allocating points, consider the level of effort and contributions for anything relevant to the team project. Note equal participation is not the same thing as equal lines of code produced. If your team used pair programming with dedicated pairs (not switching off), it is ok to allocate points to pairs rather than individuals. If some teammate(s) did a lot during the first iteration but relatively little during the second iteration, or vice versa, please tell us about it. It is very important that you *explain* the scores you give to each student, including yourself: **If you only provide numbers, without meaningful explanations, then your answer will be considered incomplete and you will receive 0 overall on the assessment.**

For example, in a team where all team members participated more or less equally, the scores would be: Sheldon 25, Leonard 25, Howard 25, Raj 25, and your explanation would describe who did which part(s)

of the overall team project effort at a conceptual level (do not tell us about individual functions or lines of code). In a team with a disparity in contributions, however, the scores might be: Homer 40, Marge 25, Bart 20, Lisa 15, and the explanation might describe Homer as doing most of the work followed by a discussion of a little bit of what the others contributed. Scores like these would most typically be provided by Homer. Marge might instead give these scores: Homer 20, Marge 30, Bart 25, Lisa 25. This is why we need every team members' explanations of who did what, and who did not do what they were expected to do, to fairly credit differential grading.

part 0B (optional): Please provide feedback on the course overall. Is there anything that you think should have been covered but wasn't? Is there anything that was covered unnecessarily (e.g., everyone already knows from their introductory courses or seems outside scope)? Is there anything that should have been discussed more, discussed less, or presented in a different way? Is there anything else you would change about the course, including contents, teaching methods, delivery, assignments, anything else you think relevant? Please explain, giving specific examples where warranted. Suggestions for next year's offering of the course will be greatly appreciated. In addition to submitting comments and suggestions here as part of your second assessment, you can also post anonymously on Edstem. (And please also submit the SEAS evaluation.)

Graded questions: There are 3 graded questions, parts 1, 2 and 3 for 45 points, 40 points, and 15 points, respectively.

Important note: Do not write or modify any code for this assessment, just think about what code changes might be needed, do not actually do it! Include in your answer a link to your team project github repository but do not change anything in your team's repository in response to this assessment. Make sure your repo is public or, if private, the instructor and your team's Mentor have collaborator access. Your Mentor will grade the assessments for everyone in your team.

Part 1 (45 points total = 15 points for each of 3 endpoints):

Pick three (3) API entry points from the main branch of your team's service (not client!) where the code implementing that entry point includes conditional branches and/or loops.

part 1A (15 points = 5 points for each of the 3 endpoints):

Write at least one user story for each of the three API entry points. Your user stories should be written in a conventional format but do not need to fit on 3x5 cards. Make sure to clarify which user story(ies) is(are) for which endpoint, and remember that the "user" in a user story can be client code or even an environment resource, not necessarily a human user. Make sure to include all relevant conditions of satisfaction for each user story.

part 1B (15 points = 5 points for each of the 3 endpoints):

Expand the user stories from part 1A to the corresponding use cases. Your use cases should be written in a conventional format. Make sure to clarify which use case(s) go with each user story. Every use

case should include at least one alternative or exception flow in addition to the basic flow (this is why we asked you to consider entry points that involve conditionals and/or loops). Include triggers, preconditions and/or postconditions as applicable.

part 1C (15 points = 5 points for each of the 3 endpoints):

For each of your three endpoints, write one or more CRC cards based on the information in your user stories and/or use cases. Your CRC cards should be written in a conventional format but do not need to fit on 3x5 cards. Make sure to clarify which CRC card(s) go with which API entry point, and that there is a CRC card for every collaborator. Note it may be possible for all three endpoints to be defined by the same CRC card and/or collaborate with the same CRC card.

Part 2 (40 points total):

Try to find code in your team's codebase, either service or client, that violates one or more of the SOLIDD design principles. The extra D is intended to include DRY as well as Dependency Inversion. If you are able to find such code, then do part A; if there is no such code in your team's codebase, then do part B. Please do only one of A or B, since only one will count. Then do part C.

part 2A (20 points, do not also do part B):

Identify which specific class (or set of classes) in your team's codebase violates one of the SOLIDD principles. Explain which principle is violated and in what way the class (possibly in conjunction with other classes) violates the principle. Then describe in detail what specific changes would be needed to this part of the code to be in compliance with SOLIDD. Do not really make any changes in your team's codebase.

part 2B (20 points, do not also do part A):

Identify a specific class (or set of classes) in your team's codebase that could intentionally be changed to violate one of the SOLIDD principles. Explain which principle you are targeting and what changes you would need to make to intentionally violate it. Do not make any actual changes in your team's codebase.

Part 2C (20 points in addition to either A or B):

Identify specific tests that already exist in your codebase that would need to be changed, or new tests that would need to be added, to unit test the code changes you described in part A or B above. Explain what specific functions and lines of code the changed or new tests would be intended to exercise that would not be exercised with your team's existing test suite. Do not make any actual changes in your team's codebase.

Part 3 (15 points):

Did your team use any of the 23 GoF design patterns (e.g., listed at

<https://www.digitalocean.com/community/tutorials/gangs-of-four-gof-design-patterns> 

9/29/26, 3:28 PMSecond Individual Assessment

(<https://www.digitalocean.com/community/tutorials/gangs-of-four-gof-design-patterns>) anywhere in your codebase, as part of either service or client? If so, describe which specific design pattern, pinpoint the exact code where it was used (e.g., file and line numbers), and describe how and why it was used. If you used more than one design pattern, just pick any one of them for this question. If you did not use any of the 23 GoF design patterns, find a part of your team’s codebase where you could have used one and describe in detail (with a specific file and line numbers) how the code would need to change to fulfill this design pattern. Do not change your team’s real codebase.

This is the end of the assessment.

Points100

Submittinga file upload

File Typespdf

Due	For	Available from	Until
Dec 9, 2024	Everyone	Dec 6, 2024 at 12:01am	Dec 12, 2024 at 11:59pm

+ Create Rubric

🔍 Find Rubric

Second Assessment

Each rubric item has a corresponding code. If your grade is not 100, there should be codes in the comments to indicate the violations. Staff/Leaders do not comment codes worth -0.

Table A. Submission Wide Deductions - Total 0 points

Code	Points	Description
A1	-100	No submission
A2	Grade x 0.75	Submitted up to 1 day late
A3	Grade x 0.5	Submitted up to 2 days late
A4	Grade x 0.25	Submitted up to 3 days late
A5	-0	Submitted On Time

Table B. Problem 1A - Total 15 points 5 per endpoint

Code	Points	Description
B1	-5	Nothing submitted per endpoint
B2	-3	User story does not fully adhere to the conventional format
B3	-1	Unclear which user story corresponds to which specific API endpoint
B4	-1	Incomplete description of conditions of satisfaction
B5	-0	1A Correct per endpoint

Table C. Problem 1B - Total 15 points 5 per endpoint

Code	Points	Description
C1	-5	Nothing submitted per endpoint
C2	-2	Use case does not follow a conventional use case format
C3	-1	Unclear how use case links to the corresponding user story from part 1A
C4	-1	Missing either trigger, preconditions, or postconditions

C5	-1	Does not include at least one alternative or exception flow
C6	-0	1B Correct per endpoint

Table D. Problem 1C - Total 15 points 5 per endpoint

Code	Points	Description
D1	-5	Nothing submitted per endpoint
D2	-1	CRC cards do not follow a conventional CRC card format
D3	-1	Unclear which CRC card(s) correspond to specific API entry points
D4	-1	Missing identification of collaborators
D5	-1	Incomplete responsibilities for the class(es)
D6	-1	Missing CRC card for collaborators
D7	-0	1C Correct per endpoint

Table E. Problem 2A - Total 20 points (if used skip Table F)

Code	Points	Description
E1	-20	Nothing submitted for Question
E2	-7	Missing several details on the violation.
E3	-7	Missing description of changes entirely.
E4	-3	Incomplete or incorrect description of changes to comply.
E5	-2	Missing details on how the principle is violated.
E6	-0	2A Correct

Table F. Problem 2B - Total 20 points (if used skip Table E)

Code	Points	Description
F1	-20	Nothing submitted for Question

F2	-7	Lacks several details on necessary changes
F3	-5	Does not explain which principle is being targeted
F4	-5	Incorrectly thinks their code is SOLIDD compliant but otherwise correct
F5	-3	Incomplete or incorrect description of changes to comply.
F6	-0	2A Correct

Table G. Problem 2C - Total 20 points

Code	Points	Description
G1	-20	Nothing submitted for Question
G2	-7	Tests are not clearly identified or are too general.
G3	-5	Does not explain which principle is being targeted
G4	-3	Explanation lacks detail on how tests cover specific functions or lines.
G5	-3	Does not adequately describe how tests ensure no disruption to existing functionalities.
G6	-0	2C Correct

Table H. Problem 3 - Total 15 points

Code	Points	Description
H1	-15	Nothing submitted for Problem 3
H2	-5	Did not mention why the pattern was/can be used
H3	-5	Did not mention how the pattern was/can be used
H4	-3	Did not mention file or line numbers
H5	-2	Did not mention the design pattern by name
H6	-0	Problem 3 Correct